

Java 1.1 — Inner Classes

Java Version: 1.1

Year: 1997

Feature: Inner Classes

Unique Class Name: InnerClasses

1. Definition

An **inner class** is a class that is declared **inside another class or interface**.

Inner classes were introduced in **Java 1.1 (1997)**. They are useful when a class is closely related to another class and should be encapsulated within it.

An inner class can access the members of its enclosing class, including **private members**.

Basic Syntax

```
class OuterClass {  
  
    class InnerClass {  
        // Inner class members  
    }  
}
```

Here:

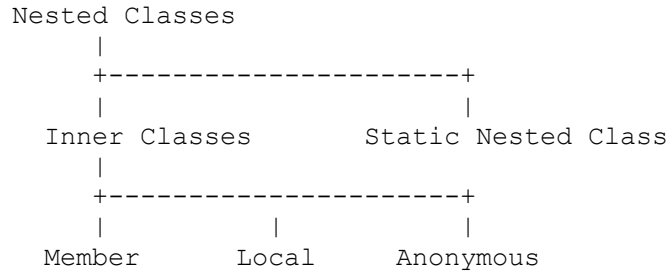
- OuterClass → outer/enclosing class
- InnerClass → inner class
- InnerClass can access members of OuterClass.

2. Types of Inner Classes

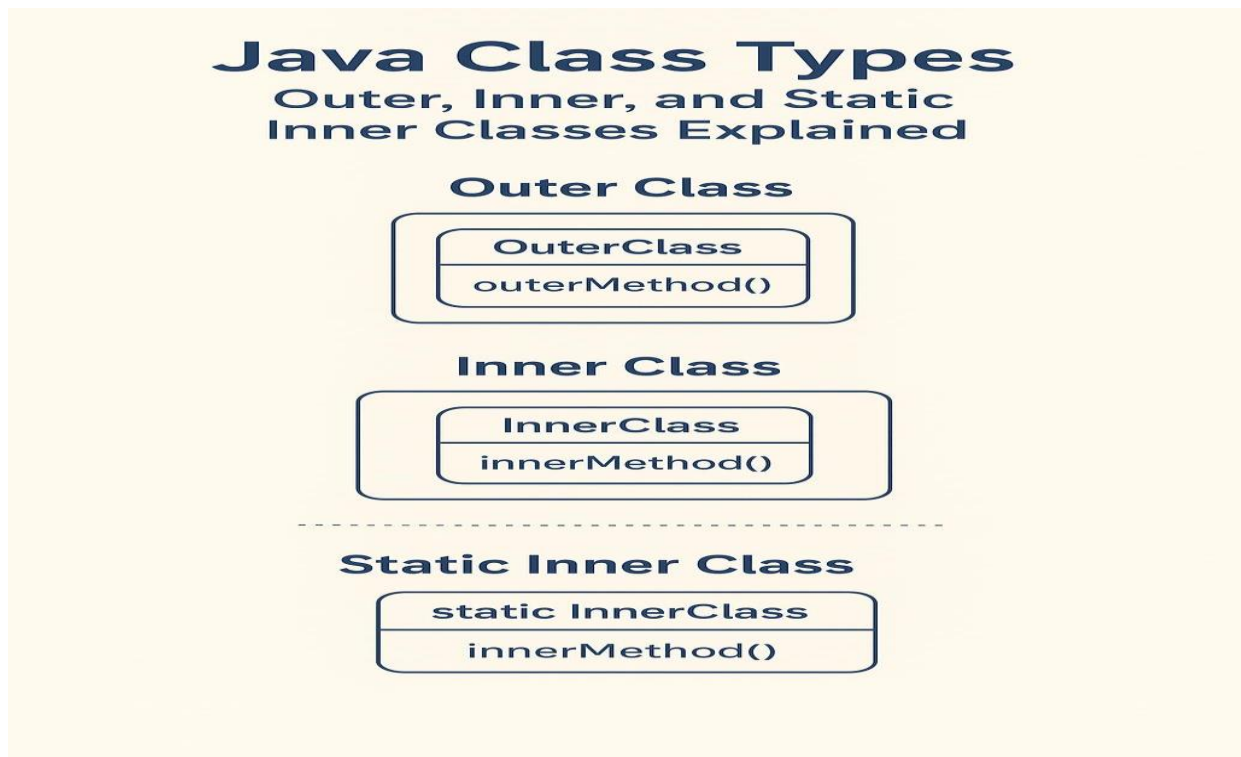
Java supports several forms of nested/inner classes:

Type	Description
Member Inner Class	Non-static class declared inside another class
Static Nested Class	static class declared inside another class
Local Inner Class	Class declared inside a method/block
Anonymous Inner Class	Class without a class name, usually used for one-time implementation

Example hierarchy:



3. Flowchart



```

class A {
    class B {
        class C {
            public void mehodOne() {
                System.out.println("Inner most method");
            }
        }
    }
}

```

```

public static void main(String ar[]) {
    A a = new A();
    A.B b = a.new B();
    A.B.C c = b.new C();
    c.mehodOne();
}

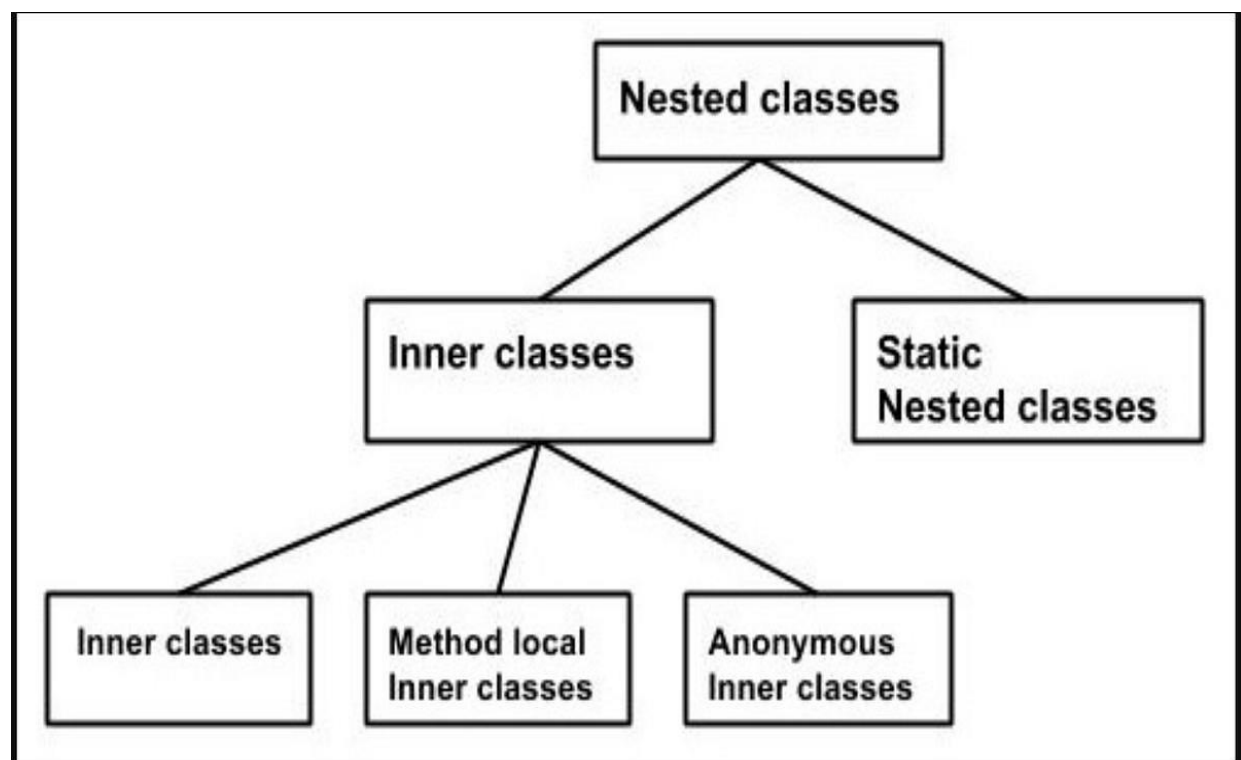
```

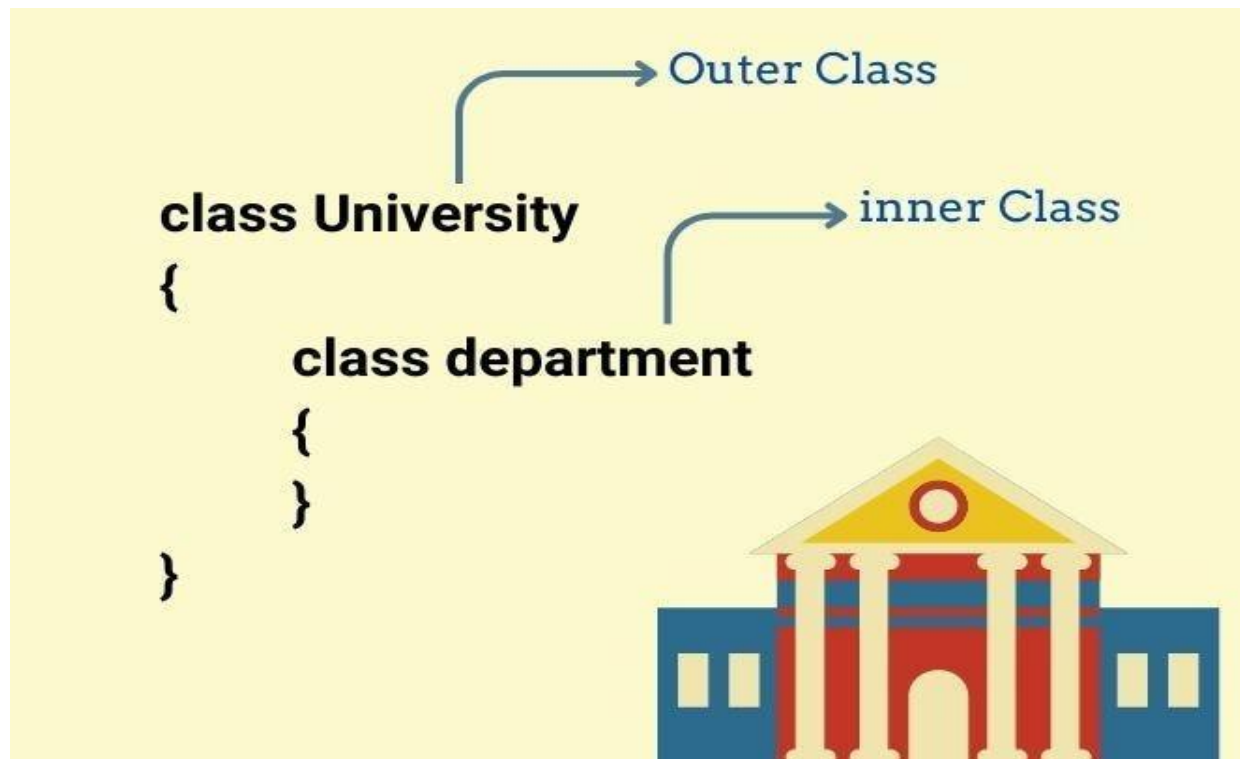
or

```

A.B b = new A().new B();
A.B.C c = new A().new B().new C();
new A().new B().new C().mehodOne();

```





Accessing Inner class code

From static area of outer class

(or)

From outside of outer class

```

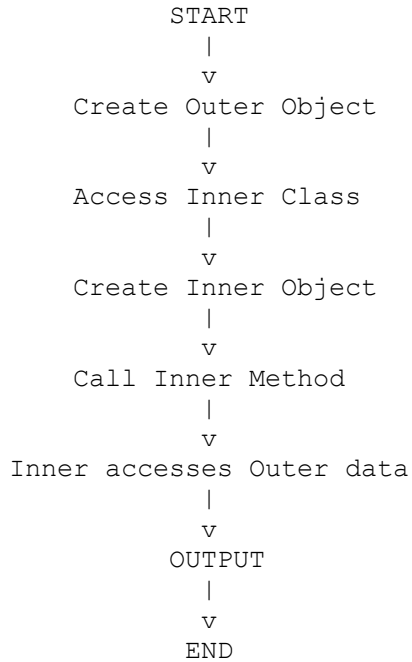
Outer o=new Outer();
Outer.Inner i=o.new Inner();
i.methodOne();
  
```

From instance area of outer class

```

Inner i=new Inner();
i.methodOne();
  
```

Program Flow



4. Java Code Example

```
public class InnerClasses {  
  
    // Outer class variable  
    private String message = "Hello from Outer Class";  
  
    // Member Inner Class  
    class InnerClass {  
  
        void display() {  
            // Inner class can access private member  
            // of the outer class  
            System.out.println(message);  
        }  
    }  
  
    public static void main(String[] args) {  
  
        // Create Outer class object  
        InnerClasses outer = new InnerClasses();  
  
        // Create Inner class object  
        InnerClasses.InnerClass inner = outer.new InnerClass();  
  
        // Call inner class method  
        inner.display();  
    }  
}
```

```
}
```

Output

Hello from Outer Class

5. How the Code Works

Step 1 — Create Outer Class Object

```
InnerClasses outer = new InnerClasses();
```

An object of the outer class is created.

```
outer  
|  
+--> message = "Hello from Outer Class"
```

Step 2 — Create Inner Class Object

```
InnerClasses.InnerClass inner = outer.new InnerClass();
```

Because InnerClass is a **non-static member inner class**, it is associated with an object of the outer class.

The general syntax is:

```
OuterClass outer = new OuterClass();  
OuterClass.InnerClass inner = outer.new InnerClass();
```

Step 3 — Call Inner Method

```
inner.display();
```

The display() method executes.

Step 4 — Access Private Member

Inside the inner class:

```
System.out.println(message);
```

Although message is declared:

```
private String message;
```

the inner class can access it because it is part of the enclosing class's scope.

6. More Practical Example

A common use of an inner class is to group helper functionality with the class that uses it.

```
public class InnerClasses {  
    private int number = 100;  
  
    class Calculator {  
        void square() {  
            int result = number * number;  
            System.out.println("Square = " + result);  
        }  
    }  
  
    public static void main(String[] args) {  
        InnerClasses outer = new InnerClasses();  
  
        InnerClasses.Calculator calculator =  
            outer.new Calculator();  
  
        calculator.square();  
    }  
}
```

Output

Square = 10000

Relationship

```
InnerClasses  
|  
|-- number = 100  
|  
+-- Calculator  
    |  
    +-- square()  
        |  
        +-- accesses number
```

7. Important Characteristics

Characteristic	Explanation
Introduced	Java 1.1
Year	1997
Location	Declared inside another class/interface
Access	Can access enclosing class members
Private access	Yes
Object dependency	Member inner class requires an outer object
Encapsulation	Helps group closely related classes
Main benefit	Better organization and encapsulation

Key Point

The most important concept to remember is:

```
Outer Class
|
+----- Inner Class
|
+----- Can access Outer Class members
```

For a **non-static member inner class**, the inner object is associated with an instance of the outer class:

```
OuterClass outer = new OuterClass();
OuterClass.InnerClass inner = outer.new InnerClass();
```

This is the fundamental syntax for understanding **Java Inner Classes introduced in Java 1.1**.

Java 1.1 — Reflection API

Java Version: 1.1

Year: 1997

Feature: Reflection API

Unique Class Name: ReflectionAPI

1. Definition

The **Reflection API** is a Java mechanism that allows a program to **inspect and interact with classes, methods, fields, constructors, and objects at runtime**, without necessarily knowing their details at compile time.

Reflection is primarily provided through the `java.lang.reflect` package and the `java.lang.Class` class.

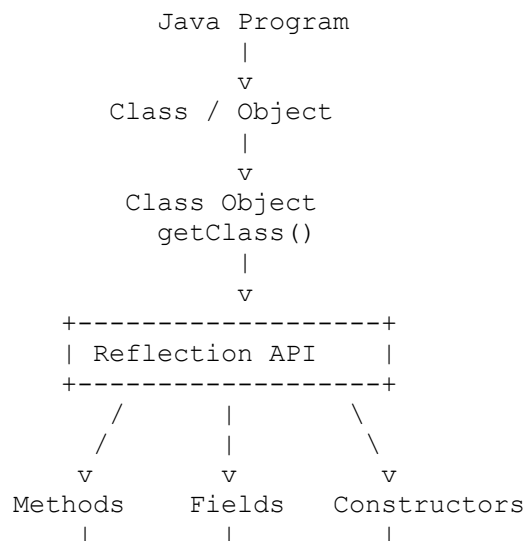
Simple Definition

Reflection is the ability of a Java program to examine and manipulate the structure and behavior of classes and objects at runtime.

For example, reflection can be used to:

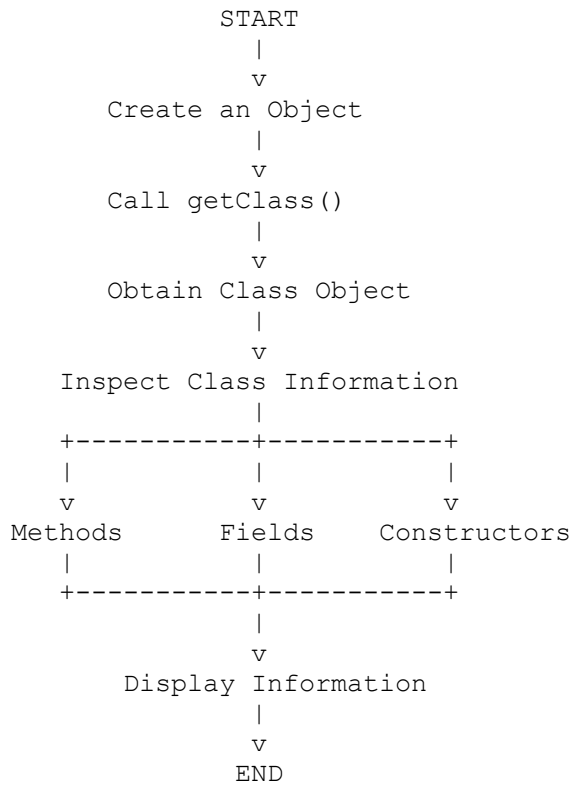
- Find a class by its name.
- Determine the class name of an object.
- Examine methods and fields.
- Examine constructors.
- Invoke methods dynamically.
- Access fields dynamically, subject to Java access-control rules.

2. Basic Reflection Architecture



v	v	v
Invoke	Read/	Create/
Method	Modify	Inspect

3. Flowchart



4. Java Code Example

```

import java.lang.reflect.Method;

public class ReflectionAPI {

    public void showMessage() {
        System.out.println("Hello from Reflection API");
    }

    public static void main(String[] args) throws Exception {

        // Create an object
        ReflectionAPI object = new ReflectionAPI();

        // Obtain Class object
        Class<?> clazz = object.getClass();
  
```

```

        // Display class information
        System.out.println("Class Name: " + clazz.getName());

        // Get all declared methods
        Method[] methods = clazz.getDeclaredMethods();

        System.out.println("\nMethods:");

        for (Method method : methods) {
            System.out.println(method.getName());
        }

        // Find a specific method
        Method method = clazz.getMethod("showMessage");

        // Invoke method dynamically
        method.invoke(object);
    }
}

```

Output

Class Name: ReflectionAPI

Methods:

main

showMessage

Hello from Reflection API

The order in which reflection returns methods is not something your program should rely on, so the method listing may appear in a different order.

5. How the Code Works

Step 1 — Create Object

```
ReflectionAPI object = new ReflectionAPI();
```

An object of ReflectionAPI is created.

```

object
  |
  v
ReflectionAPI object
  |
  +-- showMessage()

```

Step 2 — Obtain Class Object

```
Class<?> clazz = object.getClass();
```

Every Java object can provide information about its runtime class through `getClass()`.

```
Object
|
|  getClass()
v
Class object
```

Step 3 — Get Methods

```
Method[] methods = clazz.getDeclaredMethods();
```

Reflection examines the methods declared by the class.

For example:

```
ReflectionAPI
|
+-- main()
|
+-- showMessage()
```

Step 4 — Find a Specific Method

```
Method method = clazz.getMethod("showMessage");
```

The program dynamically obtains the `showMessage()` method.

Step 5 — Invoke the Method

```
method.invoke(object);
```

Instead of directly writing:

```
object.showMessage();
```

the method is invoked dynamically through reflection.

```
Method Object
|
|  invoke(object)
v
showMessage()
|
v
Hello from Reflection API
```

6. Important Reflection Classes

The Reflection API provides several important classes and interfaces.

Class / Interface	Purpose
Class	Represents a class or interface at runtime
Method	Represents a method
Field	Represents a field
Constructor	Represents a constructor
Modifier	Provides information about access modifiers
Array	Provides operations for arrays dynamically

These are mainly found in:

java.lang.reflect

7. Reflection Example with Fields

```
import java.lang.reflect.Field;

public class ReflectionAPI {

    private String name = "Anurag";

    public static void main(String[] args) throws Exception {

        ReflectionAPI object = new ReflectionAPI();

        Class<?> clazz = object.getClass();

        Field field = clazz.getDeclaredField("name");

        System.out.println("Field Name: " + field.getName());
        System.out.println("Field Type: " + field.getType().getName());
    }
}
```

Output

Field Name: name
Field Type: java.lang.String

8. Reflection Example with Constructor

```
import java.lang.reflect.Constructor;

public class ReflectionAPI {

    public ReflectionAPI() {
        System.out.println("Constructor executed");
    }

    public static void main(String[] args) throws Exception {

        Class<?> clazz = ReflectionAPI.class;

        Constructor<?> constructor =
            clazz.getConstructor();

        ReflectionAPI object =
            (ReflectionAPI) constructor.newInstance();
    }
}
```

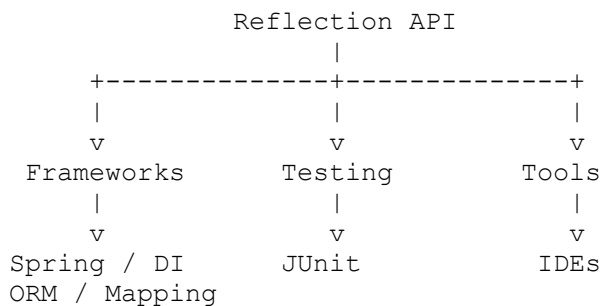
Output

Constructor executed

Here, the constructor is obtained and used dynamically.

9. Real-World Uses of Reflection

Reflection became particularly important for frameworks and tools that need to discover program structure dynamically.



Common uses include:

- **Dependency Injection**
- **Framework development**
- **ORM frameworks**
- **Serialization/deserialization**
- **Testing frameworks**
- **Plugin systems**
- **JavaBeans/introspection**
- **Runtime class discovery**
- **Dynamic method invocation**

10. Reflection vs Normal Java Code

Normal Code	Reflection
Method known at compile time	Method can be discovered at runtime
object.showMessage()	method.invoke(object)
Compile-time type information	Runtime type information
Generally simpler	More dynamic
Usually easier to understand	More complex
Usually better for ordinary application code	Useful for frameworks/tools

Normal Invocation

```
object.showMessage();
```

Reflection Invocation

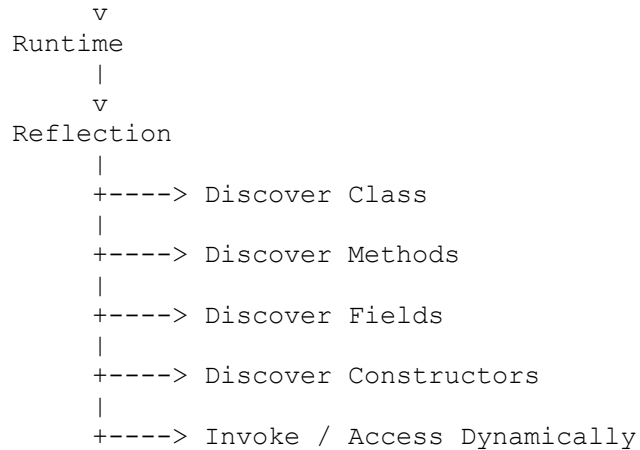
```
Method method = object.getClass()
    .getMethod("showMessage");
```

```
method.invoke(object);
```

Key Concept

The easiest way to remember the **Reflection API** is:

```
Compile Time
|
|   Class normally known
```



In one sentence:

Java Reflection allows a program to inspect and work with classes and their members dynamically at runtime.

Java 1.1 — Serialization

Java Version: 1.1

Year: 1997

Feature: Serialization

Unique Class Name: Serialization

1. Definition

Serialization is the process of converting a Java object's **state into a byte stream** so that it can be stored or transmitted.

The reverse process is called **deserialization**, where the byte stream is converted back into an object.

Simple Definition

Serialization = Object → Byte Stream

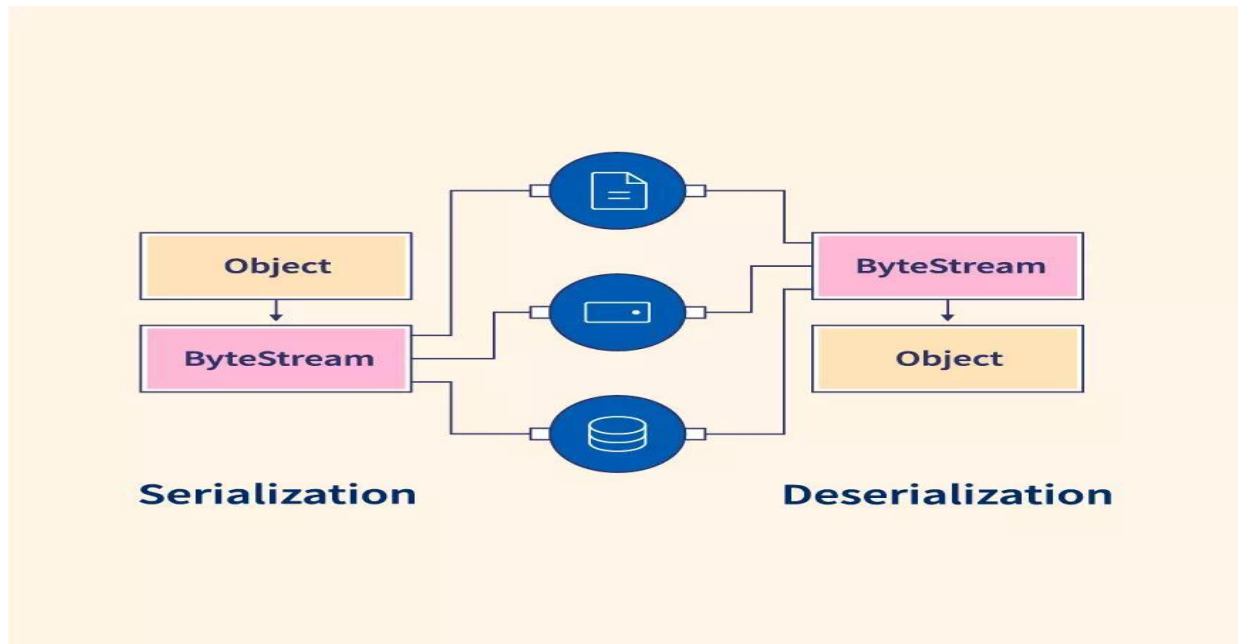
Deserialization = Byte Stream → Object

Java provides serialization primarily through:

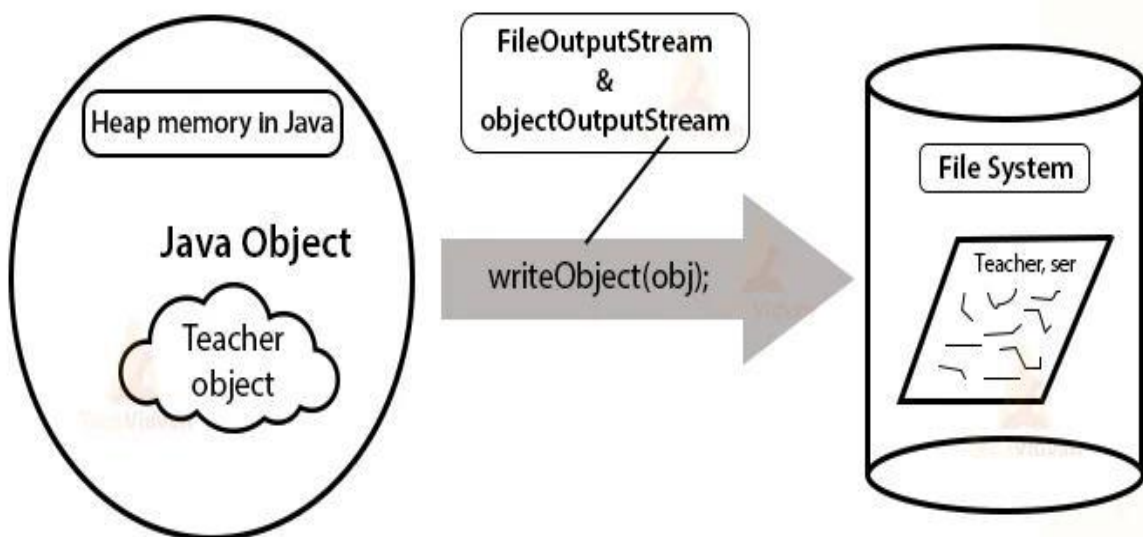
java.io.Serializable

Serializable is a **marker interface**, meaning it does not require implementing any methods.

2. Serialization Flowchart



Serialization in Java



Java Serialization

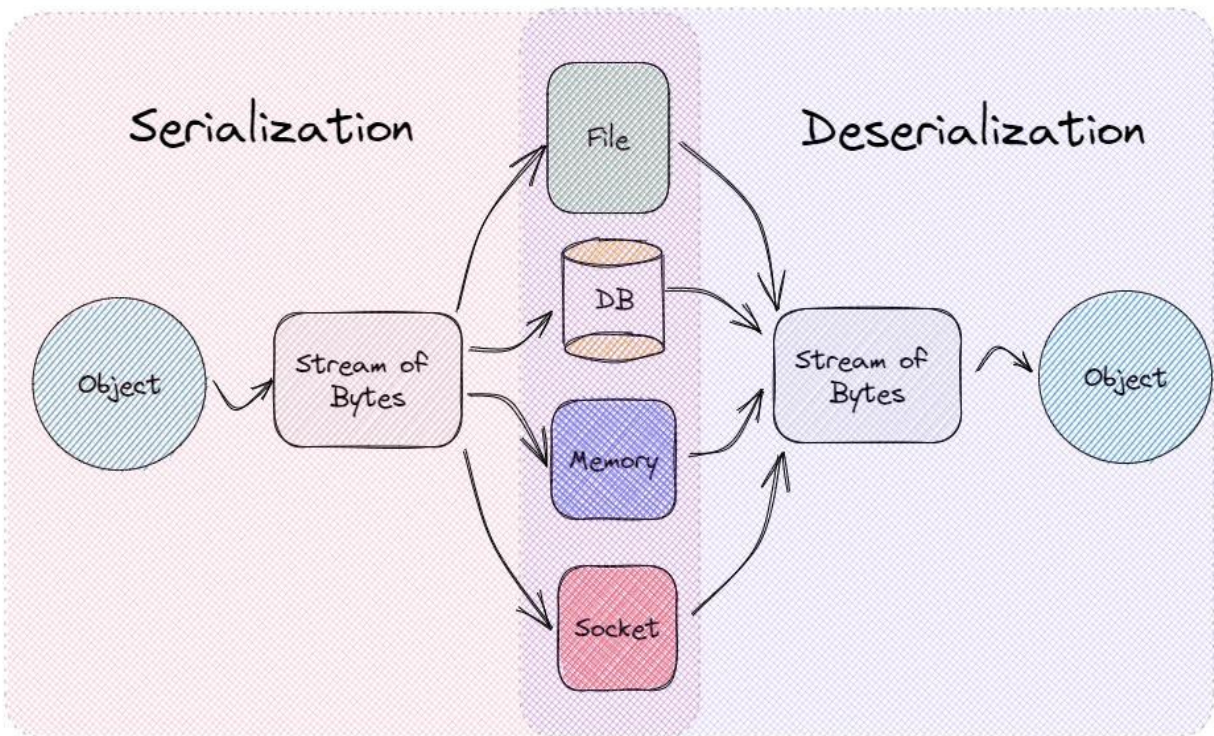
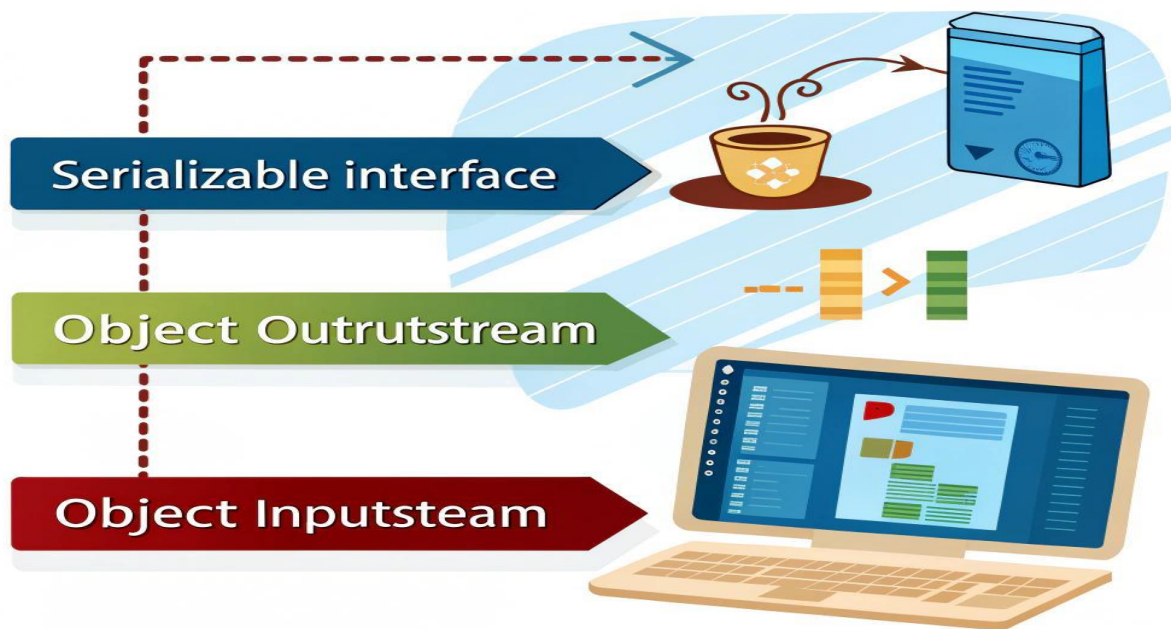
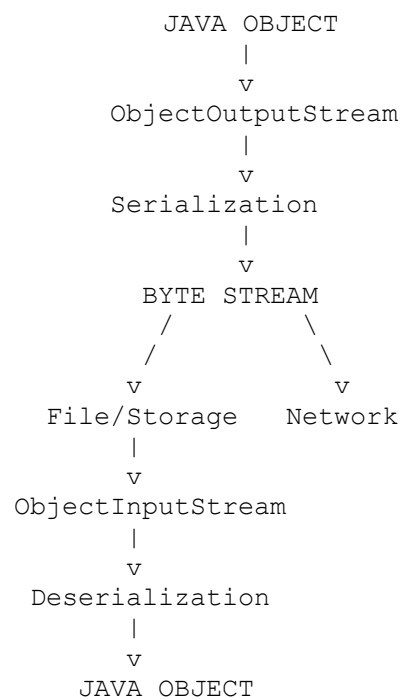
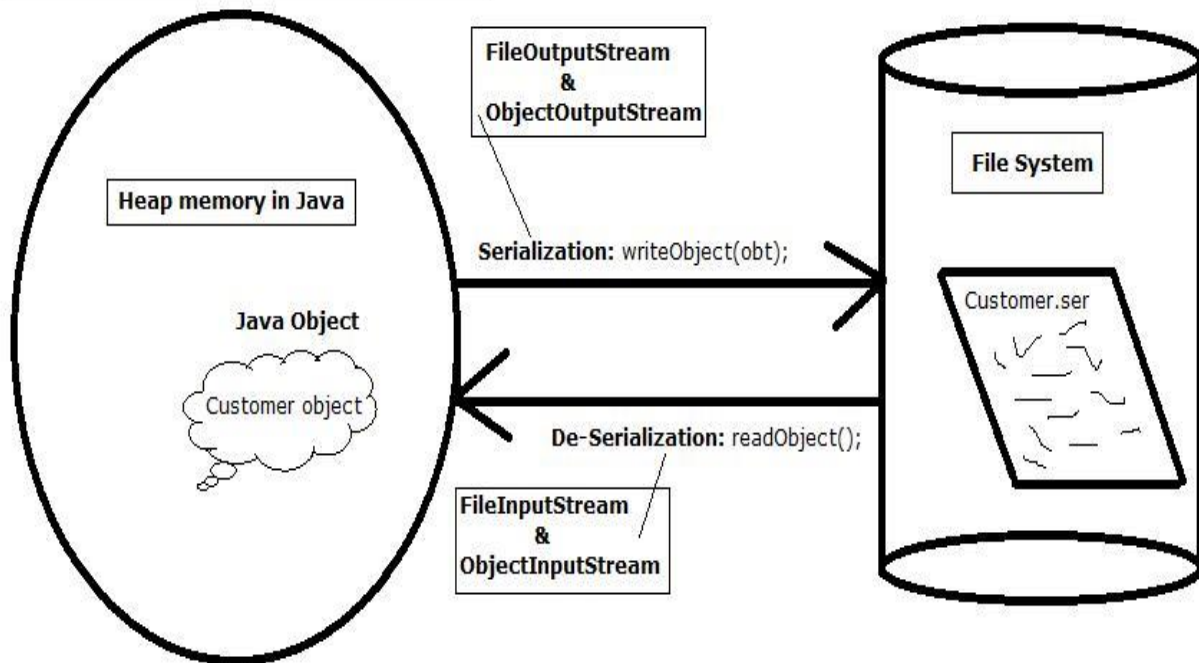


Figure: Serialization and De-Serialization in Java



3. Basic Serialization Process

Serialization:

```
Object
  |
  v
ObjectOutputStream
  |
  v
Byte Stream
  |
  v
File / Network / Storage
```

Deserialization:

```
File / Network / Storage
  |
  v
Byte Stream
  |
  v
ObjectInputStream
  |
  v
Object
```

4. Java Code Example

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;

// Serializable class
class Student implements Serializable {

    private static final long serialVersionUID = 1L;

    private int id;
    private String name;
    private String course;

    public Student(int id, String name, String course) {
        this.id = id;
        this.name = name;
        this.course = course;
    }

    public void display() {
        System.out.println("ID      : " + id);
```

```

        System.out.println("Name    : " + name);
        System.out.println("Course  : " + course);
    }
}

public class Serialization {

    public static void main(String[] args) {

        String fileName = "student.ser";

        // -----
        // Serialization
        // -----

        Student student =
            new Student(101, "Anurag", "Java");

        try (ObjectOutputStream output =
            new ObjectOutputStream(
                new FileOutputStream(fileName))) {

            output.writeObject(student);

            System.out.println("Object serialized successfully.");

        } catch (Exception e) {
            e.printStackTrace();
        }

        // -----
        // Deserialization
        // -----

        try (ObjectInputStream input =
            new ObjectInputStream(
                new FileInputStream(fileName))) {

            Student restoredStudent =
                (Student) input.readObject();

            System.out.println("\nObject deserialized successfully.");

            restoredStudent.display();

        } catch (Exception e) {
            e.printStackTrace();
        }

    }
}

```

Output

Object serialized successfully.

Object deserialized successfully.

ID : 101
Name : Anurag
Course : Java

5. How Serialization Works

Step 1 — Implement Serializable

class Student implements Serializable

This tells Java that objects of Student are eligible for default serialization.

Step 2 — Create Object

```
Student student =  
    new Student(101, "Anurag", "Java");
```

The object exists in memory:

```
student  
|  
+-- id      = 101  
+-- name    = "Anurag"  
+-- course  = "Java"
```

Step 3 — Create ObjectOutputStream

```
ObjectOutputStream output =  
    new ObjectOutputStream(  
        new FileOutputStream("student.ser"));
```

The stream provides a mechanism for writing an object to a file.

Step 4 — Serialize the Object

```
output.writeObject(student);
```

The object is converted into a byte stream.

Student Object

```
    |
    v
writeObject()
    |
    v
Byte Stream
    |
    v
student.ser
```

6. Deserialization

The reverse operation uses `ObjectInputStream`.

```
ObjectInputStream input =
    new ObjectInputStream(
        new FileInputStream("student.ser"));
```

Then:

```
Student restoredStudent =
    (Student) input.readObject();
```

The byte stream is converted back into a Java object.

```
student.ser
    |
    v
Byte Stream
    |
    v
readObject()
    |
    v
Student Object
```

7. serialVersionUID

You will commonly see:

```
private static final long serialVersionUID = 1L;
```

It identifies the serialized class version.

Example:

```
class Student implements Serializable {  
  
    private static final long serialVersionUID = 1L;  
}
```

It helps Java determine whether the serialized data is compatible with the current class definition during deserialization.

8. transient Keyword

A field marked transient is **not serialized** by Java's default serialization mechanism.

Example:

```
class Student implements Serializable {  
  
    private int id;  
    private String name;  
  
    private transient String password;  
}
```

Here:

```
id          → Serialized  
name        → Serialized  
password    → NOT serialized
```

Example:

```
class Student implements Serializable {  
  
    private int id;  
    private String name;  
    private transient String password;  
}
```

After deserialization, the password field receives its default value for its type, such as null for a reference type.

9. Important Classes and Interfaces

Class / Interface	Purpose
Serializable	Marks a class as serializable
ObjectOutputStream	Writes objects to a byte stream
ObjectInputStream	Reads objects from a byte stream
FileOutputStream	Writes bytes to a file
FileInputStream	Reads bytes from a file

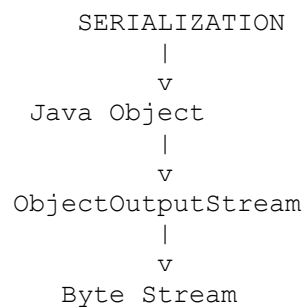
Main package:

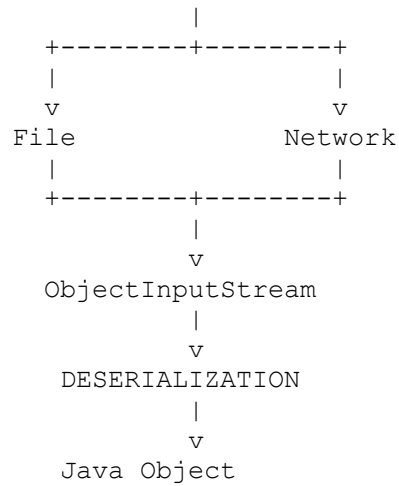
java.io

10. Serialization vs Deserialization

Serialization	Deserialization
Object → Byte Stream	Byte Stream → Object
Uses ObjectOutputStream	Uses ObjectInputStream
Uses writeObject()	Uses readObject()
Used for storing/transmitting object state	Used for reconstructing object
Example: writeObject(student)	Example: readObject()

11. Complete Concept





Key Point

Remember the two operations as:

Serialization:
Java Object —————> Byte Stream

Deserialization:
Byte Stream —————> Java Object

For example:

```
output.writeObject(student);           // Serialization
```

```
Student s =  
    (Student) input.readObject();       // Deserialization
```

Serialization was introduced in the Java 1.1 era and became an important part of java.io for persisting and transporting Java object state.

Java 1.1 — JDBC

Java Version: 1.1

Year: 1997

Feature: JDBC (Java Database Connectivity)

Unique Class Name: JDBCIntro

1. Definition

JDBC (Java Database Connectivity) is a standard Java API that allows Java applications to **connect to databases, execute SQL statements, retrieve results, and manage database operations.**

JDBC provides a common programming interface between a Java application and a relational database such as MySQL, Oracle, PostgreSQL, or SQL Server.

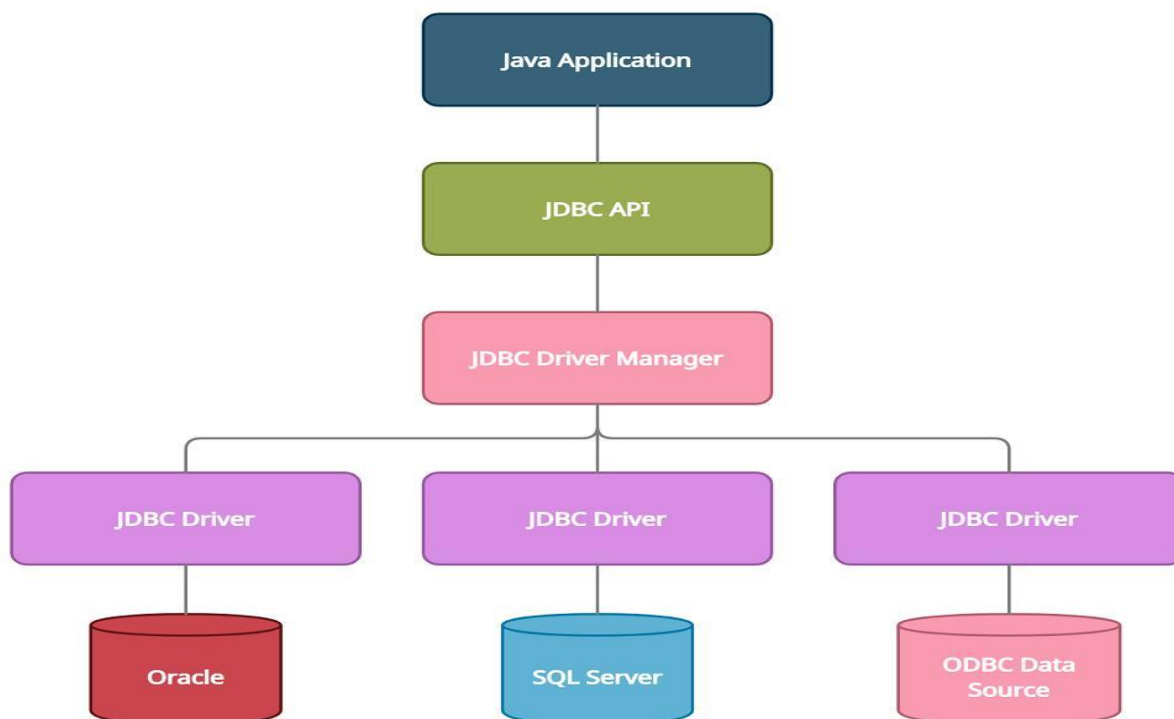
Simple Definition

JDBC is a Java API used to connect Java applications with databases and execute SQL queries.

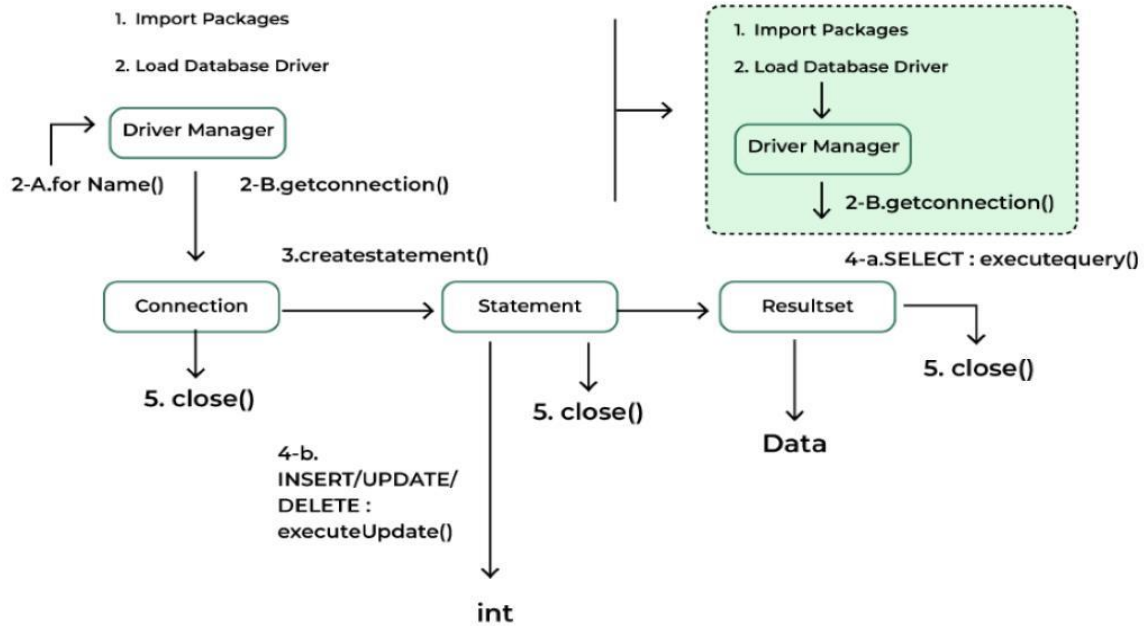
The core JDBC API is primarily located in:

java.sql

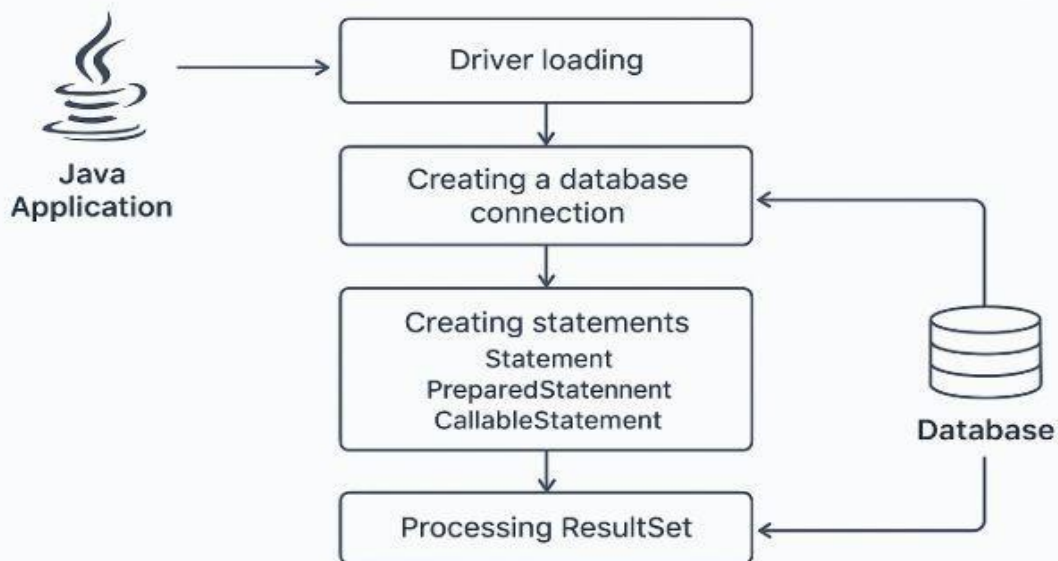
2. JDBC Architecture



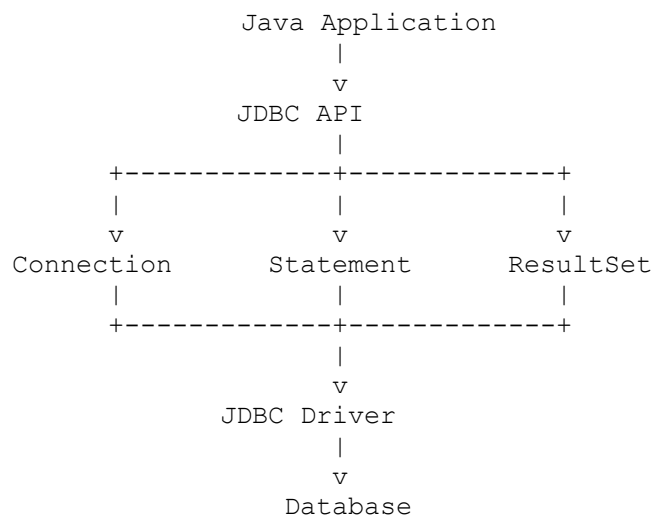
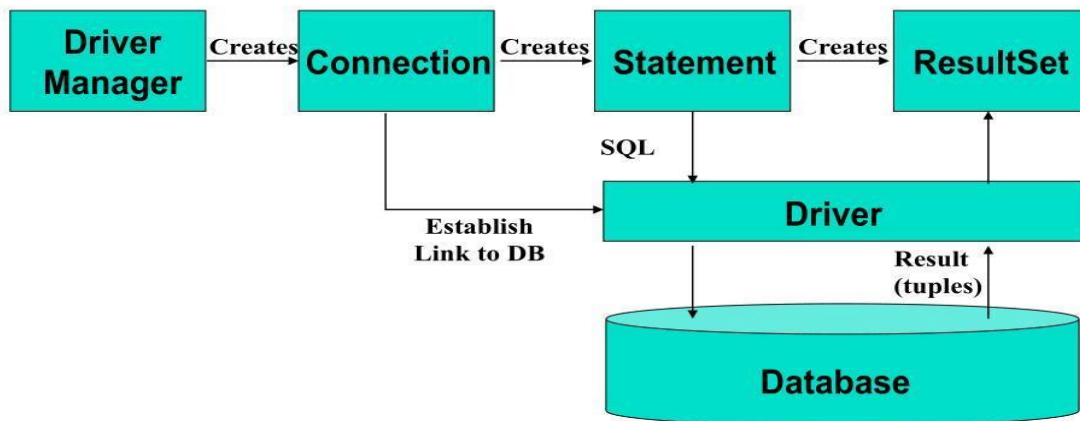
Java Database Connectivity



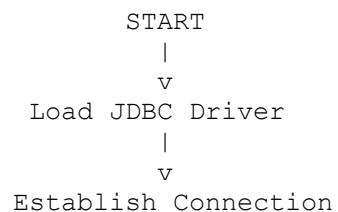
JDBC Lifecycle

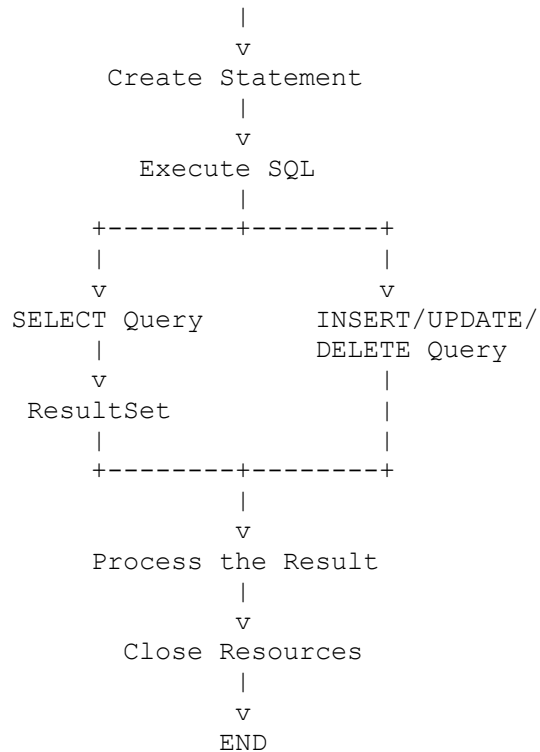


JDBC Component Interaction



3. JDBC Flowchart





4. JDBC Main Components

Component	Purpose
DriverManager	Manages JDBC drivers and establishes connections
Connection	Represents a connection to the database
Statement	Executes SQL statements
PreparedStatement	Executes parameterized SQL statements
CallableStatement	Calls stored procedures
ResultSet	Represents results returned by a query
SQLException	Represents database/JDBC errors

These are primarily part of:

java.sql

5. JDBC Code Example

The following example uses **MySQL**.

Assume a database named college contains:

```
CREATE TABLE student (  
    id INT PRIMARY KEY,  
    name VARCHAR(100),  
    course VARCHAR(100)  
);
```

Java program:

```
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.PreparedStatement;  
import java.sql.ResultSet;  
  
public class JDBCIntro {  
  
    public static void main(String[] args) {  
  
        String url =  
            "jdbc:mysql://localhost:3306/college";  
  
        String username = "root";  
        String password = "root";  
  
        String sql =  
            "SELECT id, name, course FROM student";  
  
        try {  
            Connection connection =  
                DriverManager.getConnection(  
                    url, username, password);  
  
            PreparedStatement statement =  
                connection.prepareStatement(sql);  
  
            ResultSet resultSet =  
                statement.executeQuery()  
        } {  
  
            System.out.println("Connected to database.");  
  
            while (resultSet.next()) {
```

```

        int id = resultSet.getInt("id");
        String name = resultSet.getString("name");
        String course = resultSet.getString("course");

        System.out.println(
            id + " | " + name + " | " + course);
    }

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

Example Output

Connected to database.

```

101 | Anurag | Java
102 | Rahul  | Spring Boot
103 | Amit   | Database

```

Modern JDBC drivers are normally loaded automatically through the JDBC driver mechanism, so an explicit `Class.forName()` is generally unnecessary. Older JDBC examples often used it

6. How the JDBC Code Works

Step 1 — Define Database URL

```
String url =
    "jdbc:mysql://localhost:3306/college";
```

The URL tells JDBC where the database is located.

Breakdown:

```

jdbc:mysql://localhost:3306/college
|         |         |         |         |
|         |         |         |         +-- Database
|         |         |         +----- Port
|         |         +----- Server
|         +----- Database type
+----- JDBC

```

Step 2 — Establish Connection

```
Connection connection =  
    DriverManager.getConnection(  
        url, username, password);
```

The Java application requests a connection to the database.

Java Application

|

v

DriverManager

|

v

JDBC Driver

|

v

MySQL Database

Step 3 — Create SQL Statement

```
PreparedStatement statement =  
    connection.prepareStatement(sql);
```

The SQL statement is prepared for execution.

```
SELECT id, name, course FROM student
```

Step 4 — Execute Query

```
ResultSet resultSet =  
    statement.executeQuery();
```

The database executes the SQL query and returns the result.

SQL Query

|

v

Database

|

v

ResultSet

Step 5 — Read Results

```
while (resultSet.next()) {
```

Moves through the rows returned by the database.

Then:

```
int id = resultSet.getInt("id");
String name = resultSet.getString("name");
String course = resultSet.getString("course");
```

retrieves individual column values.

7. JDBC INSERT Example

JDBC can also insert data.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

public class JDBCIntro {

    public static void main(String[] args) {

        String url =
            "jdbc:mysql://localhost:3306/college";

        String username = "root";
        String password = "root";

        String sql =
            "INSERT INTO student " +
            "(id, name, course) VALUES (?, ?, ?)";

        try (
            Connection connection =
                DriverManager.getConnection(
                    url, username, password);

            PreparedStatement statement =
                connection.prepareStatement(sql)
        ) {

            statement.setInt(1, 104);
            statement.setString(2, "Suresh");
            statement.setString(3, "JDBC");

            int rows =
                statement.executeUpdate();
```

```

        System.out.println(
            rows + " row inserted.");
    } catch (Exception e) {
        e.printStackTrace();
    }
}

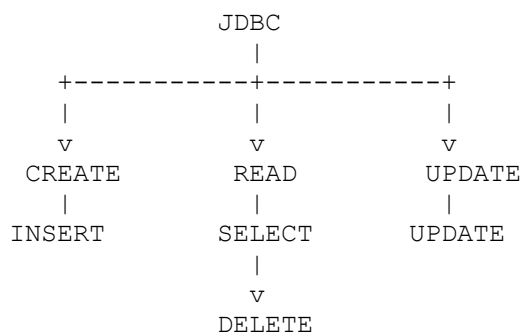
```

Output

1 row inserted.

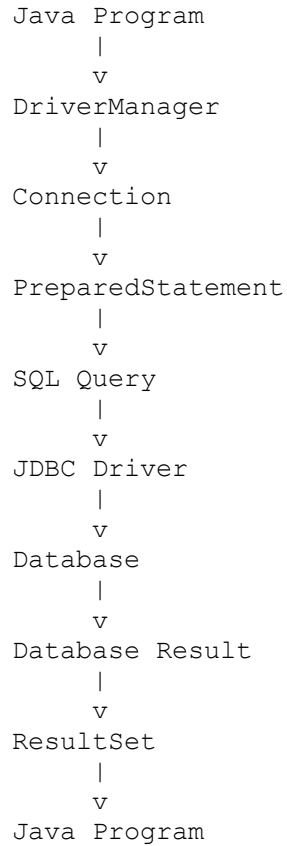
8. JDBC CRUD Operations

JDBC can implement the complete **CRUD** lifecycle:



CRUD	SQL	JDBC Method
Create	INSERT	executeUpdate()
Read	SELECT	executeQuery()
Update	UPDATE	executeUpdate()
Delete	DELETE	executeUpdate()

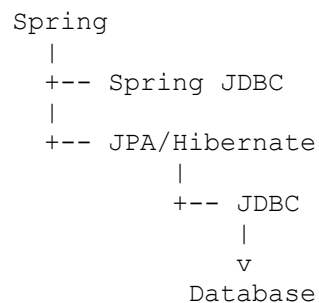
9. JDBC Execution Flow



10. Why JDBC Is Important

Before higher-level frameworks such as JPA/Hibernate and Spring Data, JDBC provided the fundamental Java-to-database programming model.

JDBC is still important because technologies such as:



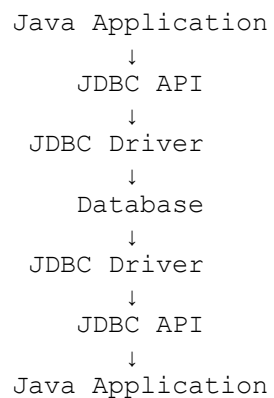
often ultimately rely on JDBC drivers and JDBC connections to communicate with relational databases.

11. Key Points to Remember

Concept	Meaning
JDBC	Java Database Connectivity
Introduced	Java 1.1 era
Package	java.sql
Connection	Connects application to database
Statement	Executes SQL
PreparedStatement	Executes parameterized SQL
ResultSet	Holds query results
DriverManager	Obtains database connections
executeQuery()	Executes queries that return rows
executeUpdate()	Executes INSERT/UPDATE/DELETE
SQLException	Handles JDBC/database errors

Key Concept

Remember JDBC as:



In one sentence:

JDBC is Java's standard database connectivity API for establishing database connections, executing SQL, processing results, and performing database operations.

Java 1.1 — JavaBeans

Java Version: 1.1

Year: 1997

Feature: JavaBeans

Unique Class Name: JavaBeans

1. Definition

JavaBeans is a Java component model introduced in **Java 1.1** for creating **reusable, modular, and configurable software components**.

A JavaBean is a Java class that follows certain conventions, such as:

- Having a **public no-argument constructor**.
- Providing properties through **getter and setter methods**.
- Generally implementing `Serializable` so the component can be persisted.
- Supporting introspection, allowing tools to discover its properties and methods.

Simple Definition

JavaBeans are reusable Java software components that follow standard conventions for properties, constructors, and events.

For example:

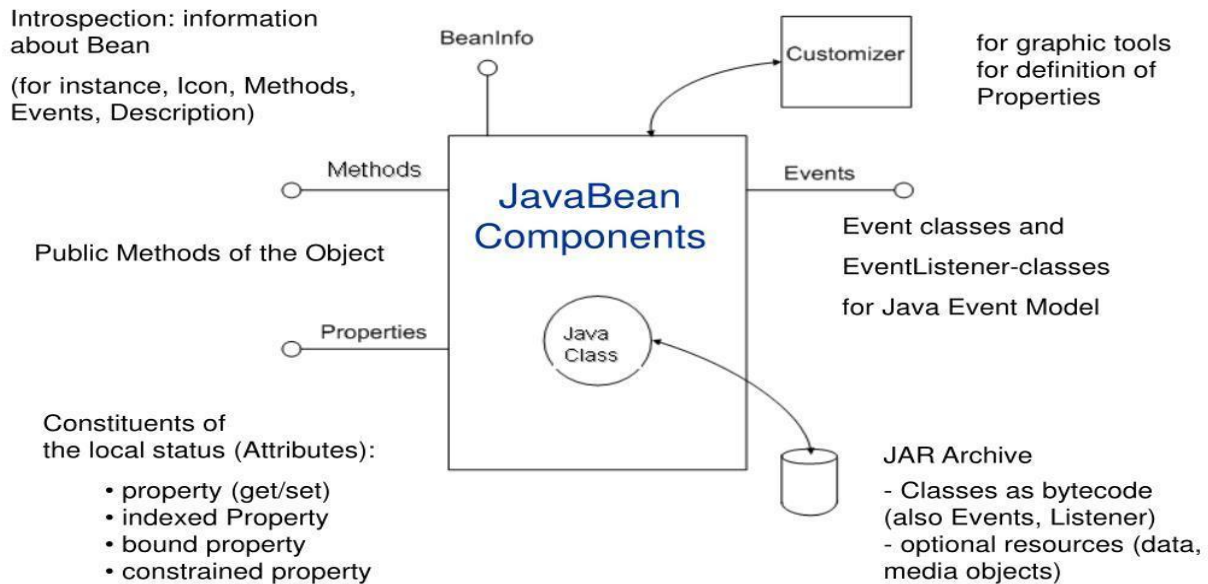
```
public class Student implements java.io.Serializable {  
  
    private String name;  
  
    public Student() {  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

Here, name is a **JavaBean property**.

2. JavaBeans Architecture

VII.4

JavaBeans: component model



Components in Java: JavaBeans

Any Java class has the capability of being a JavaBean, such as ButtonControl

- **Methods**

requestFocus ()

enable ()

- **Properties**

getLabel ()

setLabel ()

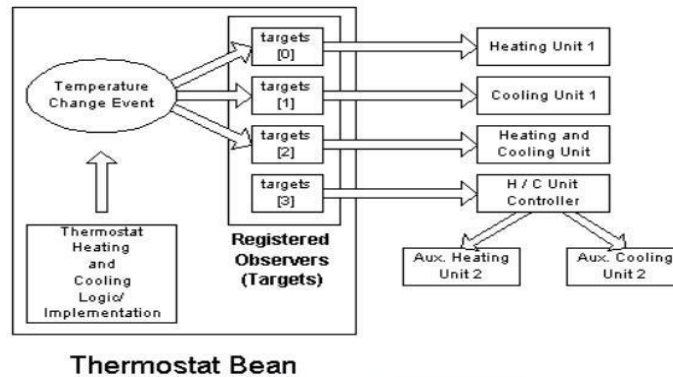
label

- **Events**

buttonControl1

Typical Bean Interactions

- Often, JavaBeans component will function as a source for certain types of events, yet be capable of registering as a target for events produced by other components.



AP 6/02

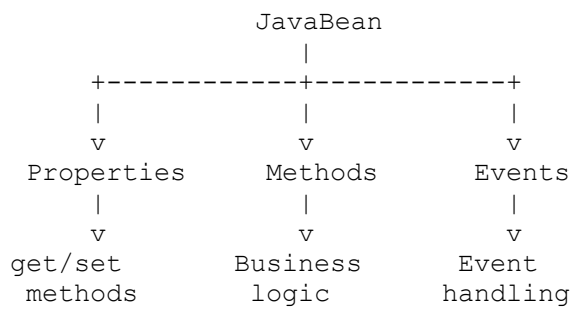
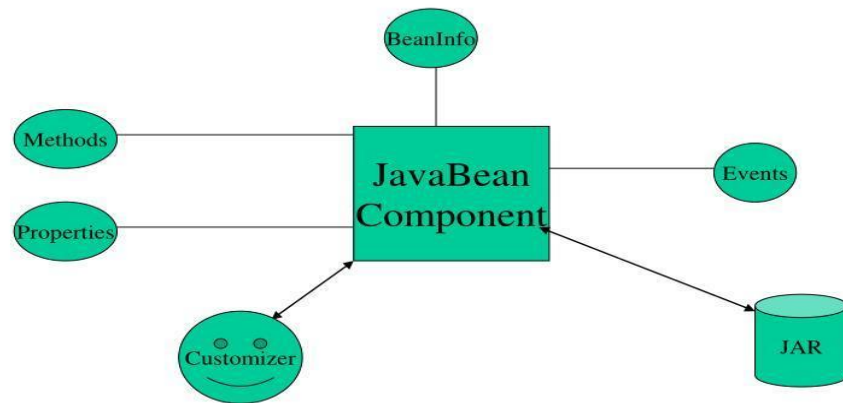
JavaBean Properties

Properties have “get” and “set” methods

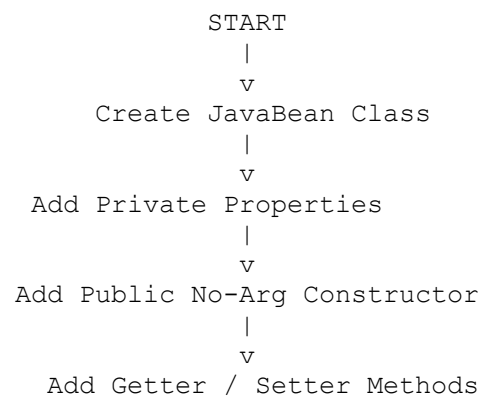
```
// Possible implementation
public class GridControl ... {
    private int columnHeaderHeight;

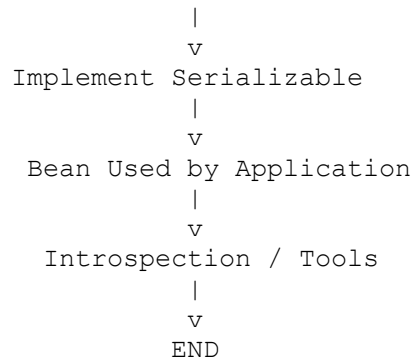
    public int getColumnHeaderHeight() {
        return columnHeaderHeight;
    }
    public void setColumnHeaderHeight(int h) {
        columnHeaderHeight = h;
    } ...
}
```



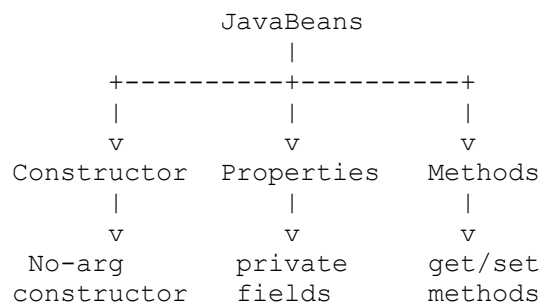


3. JavaBeans Flowchart





4. Basic JavaBean Structure



Typical structure:

```

public class JavaBeans implements Serializable {

    private String name;

    public JavaBeans() {
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}

```

5. Java Code Example

```
import java.io.Serializable;

public class JavaBeans implements Serializable {

    private static final long serialVersionUID = 1L;

    // Bean properties
    private int id;
    private String name;
    private String course;

    // Public no-argument constructor
    public JavaBeans() {
    }

    // Getter for id
    public int getId() {
        return id;
    }

    // Setter for id
    public void setId(int id) {
        this.id = id;
    }

    // Getter for name
    public String getName() {
        return name;
    }

    // Setter for name
    public void setName(String name) {
        this.name = name;
    }

    // Getter for course
    public String getCourse() {
        return course;
    }

    // Setter for course
    public void setCourse(String course) {
        this.course = course;
    }

    // Business method
    public void display() {
        System.out.println("ID      : " + id);
        System.out.println("Name    : " + name);
        System.out.println("Course : " + course);
    }

    public static void main(String[] args) {
```

```

        // Create JavaBean object
        JavaBeans student = new JavaBeans();

        // Set properties
        student.setId(101);
        student.setName("Anurag");
        student.setCourse("Java");

        // Read properties
        System.out.println("Student Name: "
            + student.getName());

        // Call business method
        student.display();
    }
}

```

Output

Student Name: Anurag

ID : 101
 Name : Anurag
 Course : Java

6. How the JavaBean Works

Step 1 — Private Properties

```

private int id;
private String name;
private String course;

```

The bean's data is kept private.

```

JavaBeans
|
+-- private id
+-- private name
+-- private course

```

This supports **encapsulation**.

Step 2 — No-Argument Constructor

```

public JavaBeans() {

```

```
}
```

A public no-argument constructor allows tools and frameworks to instantiate the bean easily.

For example:

```
JavaBeans student = new JavaBeans();
```

Step 3 — Setter Methods

```
student.setName("Anurag");
```

The setter changes the property.

```
setName("Anurag")
  |
  v
name property
  |
  v
"Anurag"
```

Step 4 — Getter Methods

```
student.getName();
```

The getter retrieves the property.

```
name property
  |
  v
getName()
  |
  v
"Anurag"
```

7. JavaBean Property Convention

JavaBeans uses naming conventions to identify properties.

For a property:

```
private String name;
```

the conventional methods are:

```
public String getName() {  
    return name;  
}
```

```
public void setName(String name) {  
    this.name = name;  
}
```

For a boolean property:

```
private boolean active;
```

the conventional getter can be:

```
public boolean isActive() {  
    return active;  
}
```

and:

```
public void setActive(boolean active) {  
    this.active = active;  
}
```

8. JavaBean vs Normal Java Class

Feature	Normal Java Class	JavaBean
Private fields	Optional	Common convention
No-arg constructor	Optional	Standard convention
Getter/setter methods	Optional	Standard property convention
Serializable	Optional	Commonly implemented
Reusable component	Possible	Specifically designed for it
Introspection	Not necessarily bean-oriented	Supported through JavaBeans conventions
Tool/framework support	General	Designed for component tools/frameworks

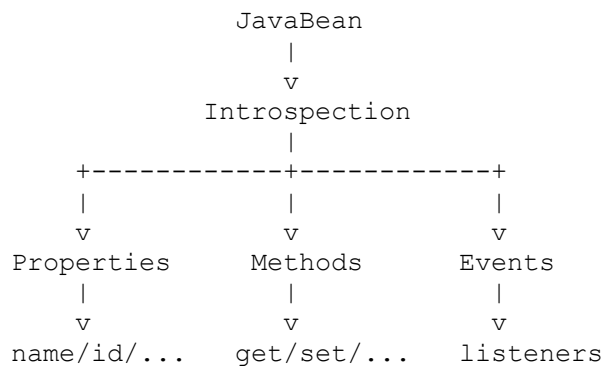
--	--	--

Important: Not every Java class is a JavaBean. A JavaBean follows the JavaBeans component conventions.

9. JavaBeans and Introspection

One important JavaBeans feature is **introspection**.

Introspection allows tools or frameworks to discover information about a bean, such as:



Java provides classes such as:

`java.beans.Introspector`
`java.beans.BeanInfo`
`java.beans.PropertyDescriptor`

For example:

```
import java.beans.BeanInfo;
import java.beans.Introspector;
import java.beans.PropertyDescriptor;

public class JavaBeans {

    private String name;

    public JavaBeans() {
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

```

public static void main(String[] args)
    throws Exception {

    BeanInfo beanInfo =
        Introspector.getBeanInfo(JavaBeans.class);

    for (PropertyDescriptor property :
        beanInfo.getPropertyDescriptors()) {

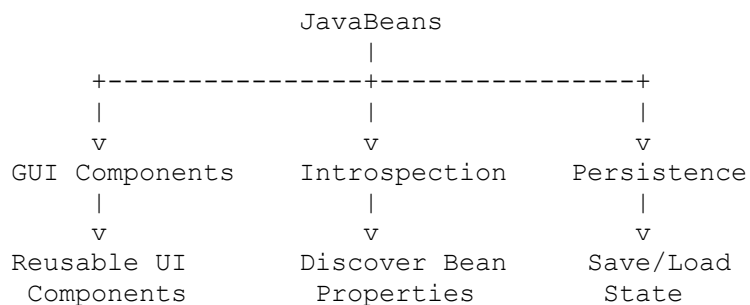
        System.out.println(
            "Property: " + property.getName());
    }
}

```

This demonstrates how JavaBeans conventions allow software tools to discover bean properties automatically.

10. JavaBeans in Real Applications

The JavaBeans component model influenced many Java technologies.



JavaBeans concepts also appear in technologies and frameworks that use Java object properties and introspection conventions.

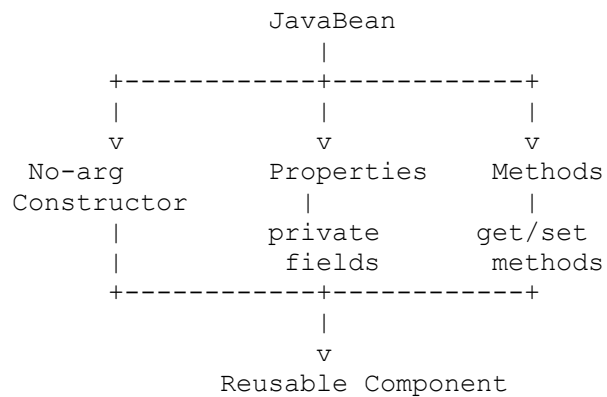
11. Key Characteristics

Characteristic	Explanation
Introduced	Java 1.1
Year	1997
Purpose	Reusable Java software components

Constructor	Public no-argument constructor is standard
Properties	Usually private fields
Access	Getter/setter methods
Persistence	Commonly implements Serializable
Introspection	Supported
Events	JavaBeans event model
Reusability	Major design goal

Key Concept

Remember JavaBeans using this pattern:



In one sentence:

JavaBeans are reusable Java components that follow standard conventions for constructors, properties, methods, events, and introspection, introduced with Java 1.1.

Java 1.1 — RMI

Java Version: 1.1

Year: 1997

Feature: RMI (Remote Method Invocation)

Unique Class Name: RMI

1. Definition

RMI (Remote Method Invocation) is a Java technology that allows an object running in one **JVM** to invoke methods on an object running in another **JVM**, potentially on a different computer.

Simple Definition

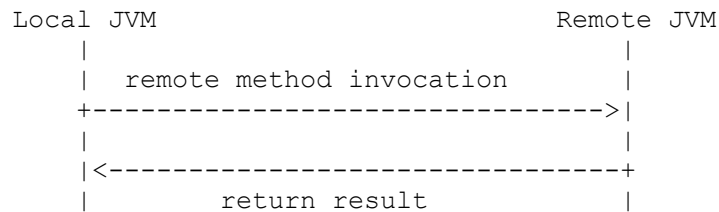
RMI allows a Java program to call methods on a remote Java object almost as if the object were local.

RMI is primarily associated with the:

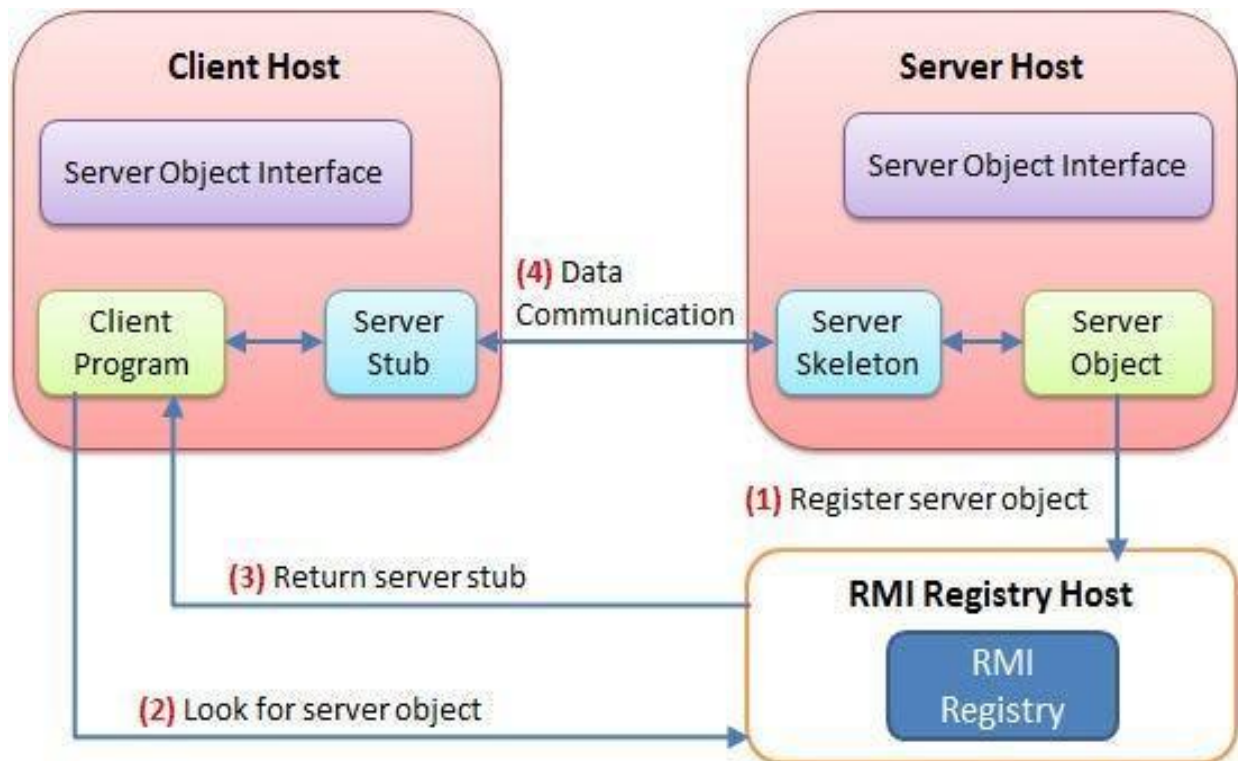
java.rmi

package.

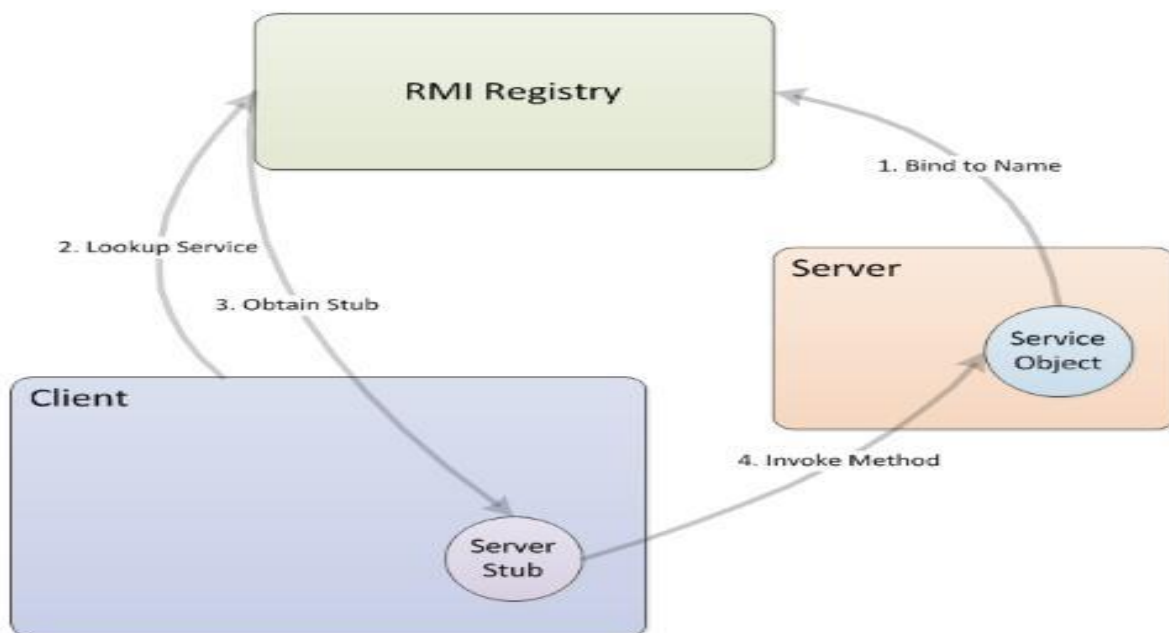
Basic Concept

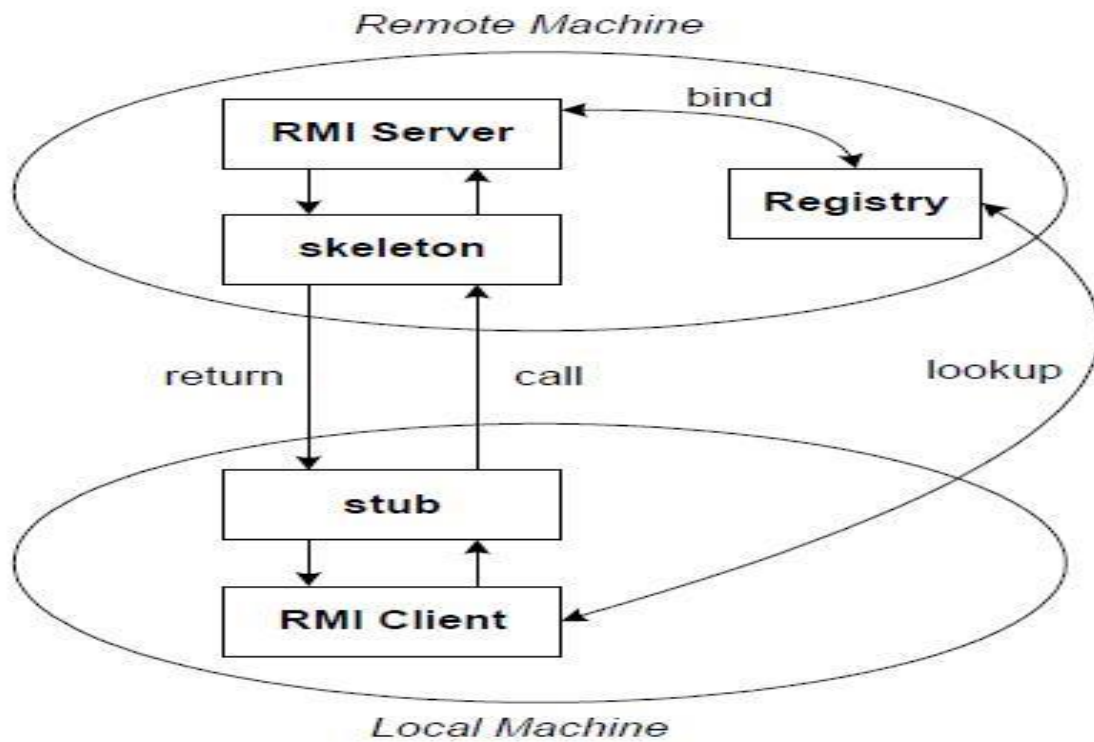


2. RMI Architecture

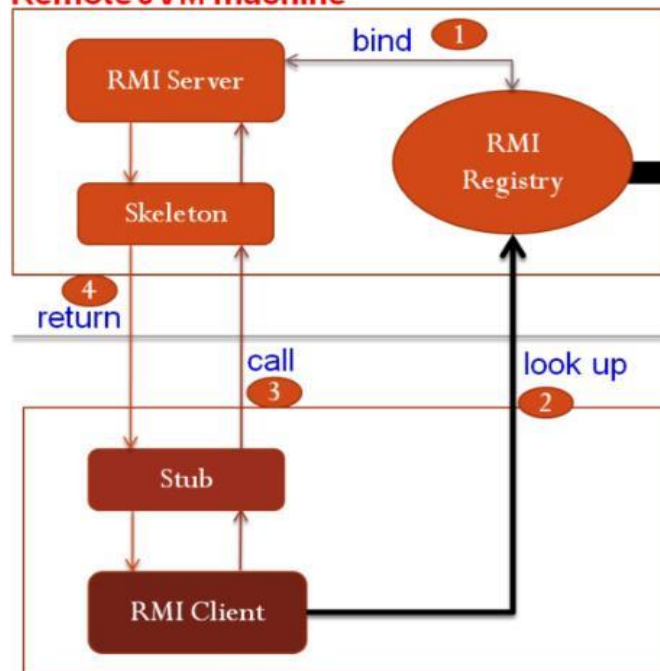


How Java RMI Works



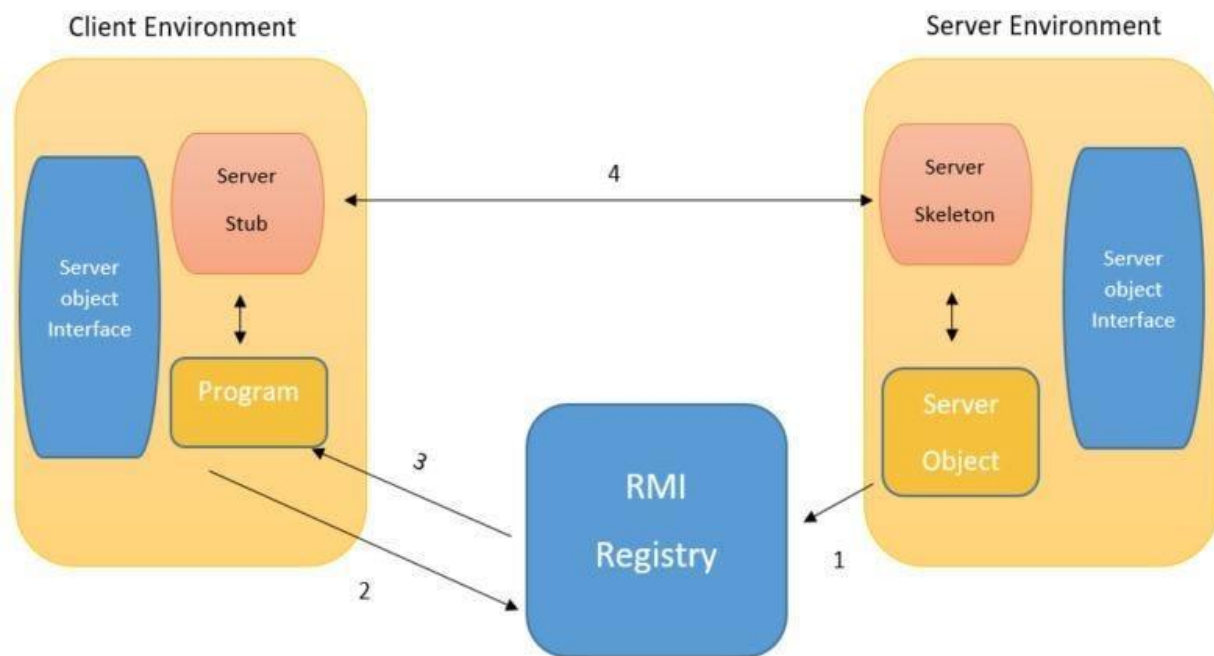


Remote JVM machine

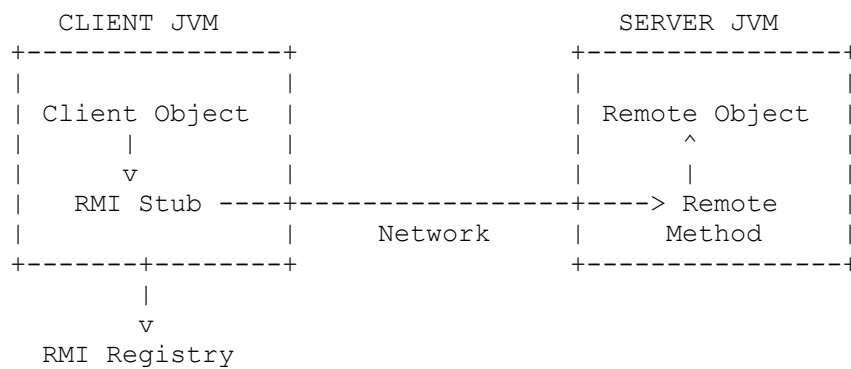


Local JVM machine





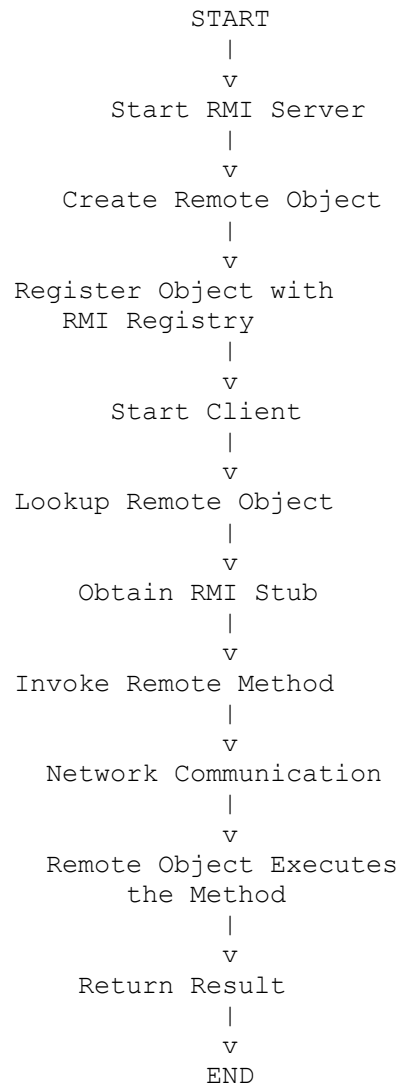
RMI Architecture



The major components are:

1. **RMI Client**
2. **Remote Interface**
3. **RMI Stub**
4. **RMI Registry**
5. **Remote Object**
6. **RMI Server**

3. RMI Flowchart



4. RMI Communication

Suppose the client wants to execute:

```
calculate.add(10, 20);
```

The actual object may exist in another JVM.

```
Client JVM
|
| add(10, 20)
```

```

      v
RMI Stub
  |
  | Network
  v
RMI Runtime
  |
  v
Remote Object
  |
  | add(10, 20)
  v
Result = 30
  |
  v
Client JVM

```

The client does not directly manipulate the remote object's memory. The RMI infrastructure handles communication between the JVMs.

5. Java RMI Example

A simple RMI application can be divided into four source files:

```

RMI Project
├── Calculator.java
├── CalculatorImpl.java
├── RMIServer.java
└── RMIClient.java

```

6. Step 1 — Remote Interface

```

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Calculator extends Remote {

    int add(int a, int b) throws RemoteException;
}

```

Explanation

The interface extends:

Remote

This indicates that its methods can be invoked remotely.

The method declares:

throws RemoteException

because remote communication can fail due to network or RMI-related problems.

7. Step 2 — Remote Object Implementation

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class CalculatorImpl
    extends UnicastRemoteObject
    implements Calculator {

    public CalculatorImpl() throws RemoteException {
        super();
    }

    @Override
    public int add(int a, int b)
        throws RemoteException {

        return a + b;
    }
}
```

Here:

extends UnicastRemoteObject

makes the object available for remote invocation using RMI's standard unicast remote-object mechanism.

8. Step 3 — RMI Server

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class RMIServer {

    public static void main(String[] args) {

        try {
            // Create remote object
```

```

        Calculator calculator =
            new CalculatorImpl();

        // Start RMI Registry on port 1099
        Registry registry =
            LocateRegistry.createRegistry(1099);

        // Bind remote object
        registry.rebind(
            "Calculator",
            calculator);

        System.out.println(
            "RMI Server is running...");

    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

Server Flow

```

RMIServer
|
v
CalculatorImpl
|
v
RMI Registry
|
v
"Calculator"
|
v
Waiting for Client

```

9. Step 4 — RMI Client

```

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class RMIClient {

    public static void main(String[] args) {

        try {
            // Connect to RMI Registry
            Registry registry =
                LocateRegistry.getRegistry(

```

```

        "localhost",
        1099);

    // Look up remote object
    Calculator calculator =
        (Calculator) registry.lookup(
            "Calculator");

    // Invoke remote method
    int result =
        calculator.add(10, 20);

    System.out.println(
        "Result = " + result);

} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

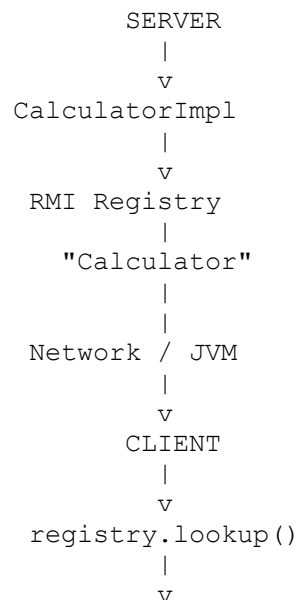
Output

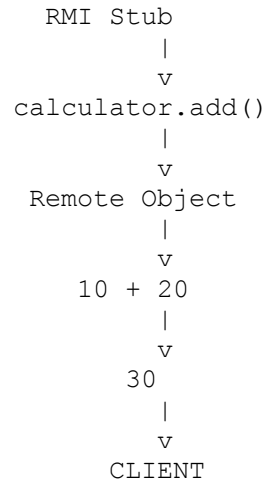
Result = 30

The server may display:

RMI Server is running...

10. Complete RMI Flow





11. Important RMI Components

Component	Purpose
Remote	Marks an interface whose methods may be invoked remotely
RemoteException	Represents communication/RMI failures
UnicastRemoteObject	Supports exporting remote objects
RMI Registry	Stores and provides references to remote objects
Remote Interface	Defines methods available remotely
Remote Implementation	Provides actual method implementation
Client	Looks up and invokes remote methods
Stub	Client-side representation used to communicate with the remote object

12. RMI Registry

The **RMI Registry** acts like a directory.

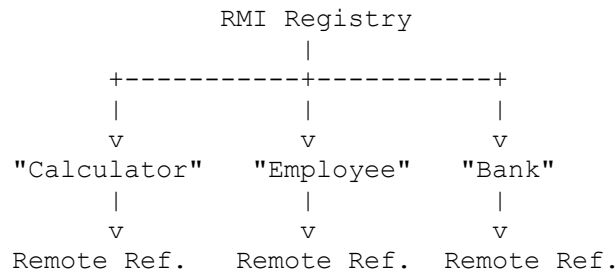
The server registers an object:

```
registry.rebind("Calculator", calculator);
```

The client searches for it:

```
Calculator calculator =  
    (Calculator) registry.lookup("Calculator");
```

Conceptually:



13. RMI vs Local Method Call

Normal Java Method Call

```
Calculator calculator = new CalculatorImpl();
```

```
int result = calculator.add(10, 20);
```

The object exists in the same JVM.

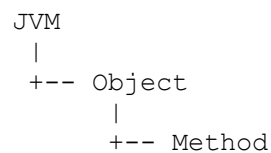
RMI Method Call

```
Calculator calculator =  
    (Calculator) registry.lookup("Calculator");
```

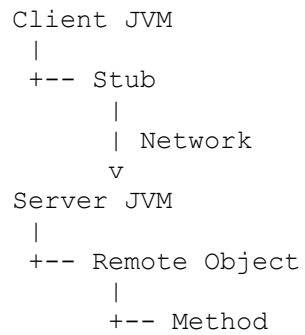
```
int result = calculator.add(10, 20);
```

The implementation can exist in another JVM.

Normal:



RMI :



14. Key Characteristics of RMI

Characteristic	Description
Full form	Remote Method Invocation
Introduced	Java 1.1
Year	1997
Package	java.rmi
Purpose	Remote Java-to-Java method invocation
Communication	Between JVMs
Registry	Locates remote objects
Remote interface	Extends Remote
Remote methods	Usually declare RemoteException
Server	Exposes remote object
Client	Looks up and invokes remote methods

Key Concept

Remember RMI as:

```
Client JVM
  |
  | 1. lookup()
  v
RMI Registry
  |
  | 2. remote reference
  v
RMI Stub
  |
  | 3. method call
  v
Network
  |
  v
Remote Object
  |
  | 4. execute method
  v
Return Result
  |
  v
Client JVM
```

In one sentence:

Java RMI allows one JVM to invoke methods on a remote Java object located in another JVM, with the RMI runtime and registry handling the communication and object lookup.